

CertiGap

Сертифицируемый компилятор адаптивных структур данных

Workload + ограничения → структура + код + проверяемое объяснение

01

НАУЧНАЯ
ЗАЩИТА

Одна структура не бывает лучшей всегда

Fenwick

быстрые обновления и суммы

Prefix sum

идеален при почти неизменных данных

Segment tree

универсален, но требует больше памяти

Learned / adaptive

выигрывает только при подходящем workload

Реальная трудность

Выбрать структуру до того, как известна будущая нагрузка, и не ошибиться при ограниченной памяти.

Сегодня это обычно делается вручную и без формального доказательства качества.

Пример: каталог с горячими и холодными ключами

80% запросов

приходятся на 5% товаров



холодная область

горячая область

холодная область

Идея CertiGap

Не тратить одинаковый бюджет на все ключи: построить более быстрые пути там, где запросы действительно сосредоточены.

Цель, гипотеза и задачи исследования

Цель

Создать систему, которая выбирает или синтезирует структуру данных под workload и выдаёт независимо проверяемый сертификат.

Гипотеза

При ограниченном бюджете и неравномерных запросах частичная адаптивная материализация лучше простых balanced и greedy решений.

Задачи

Формализовать стоимость; создать exact и scalable алгоритмы; проверить корректность; сравнить baselines; реализовать C++/Python/SQLite.

Независимая переменная: распределение операций и бюджет. Метрики: objective gap, latency, память, regret и корректность сертификата.

Формальная модель

Вход

- отсортированные ключи $x_1 \dots x_n$
- частоты запросов p_i
- бюджет B разделений
- стоимость fallback η

Оптимизация


$$\min \sum p_i \cdot \text{cost}(i, T)$$
$$T: \text{splits}(T) \leq B$$

Выход

- частичная структура T
- ожидаемая стоимость
- нижняя граница
- сертификат проверки

Ключевое отличие

Мы оптимизируем не только форму полного дерева, а какую часть порядка вообще выгодно материализовать.

Одна система вместо набора несвязанных модулей



Единый контракт

workload + допустимые операции + ограничение памяти → исполняемая структура + доказательство выбора

AutoIndex сначала рассматривает классические решения

Sorted array

точки · мало памяти

Prefix sum

статические суммы

Fenwick

суммы + обновления

Sqrt decomposition

жёсткий бюджет

Segment tree

общие агрегаты

Sparse table

статические min/max

CertiRange Point

скошенные точки

CertiRange Range

скошенные диапазоны

Если Fenwick или prefix sum лучше, система обязана выбрать их — CertiGar не навязывает собственный алгоритм.

Алгоритмическое ядро: точность там, где возможно

01	Exact frontier DP	глобальный optimum	малые/средние задачи
02	Beam search	почти exact в тестах	масштабируемый путь
03	Branch-and-Bound	upper/lower bound	anytime TV-DRO
04	Greedy	быстрый baseline	может ошибаться сильно

На малых задачах exact служит oracle; на больших beam получает проверяемый upper bound и сравнивается с lower bound

Сертификат отделяет утверждение от доверия к программе

CERTIFICATE.json

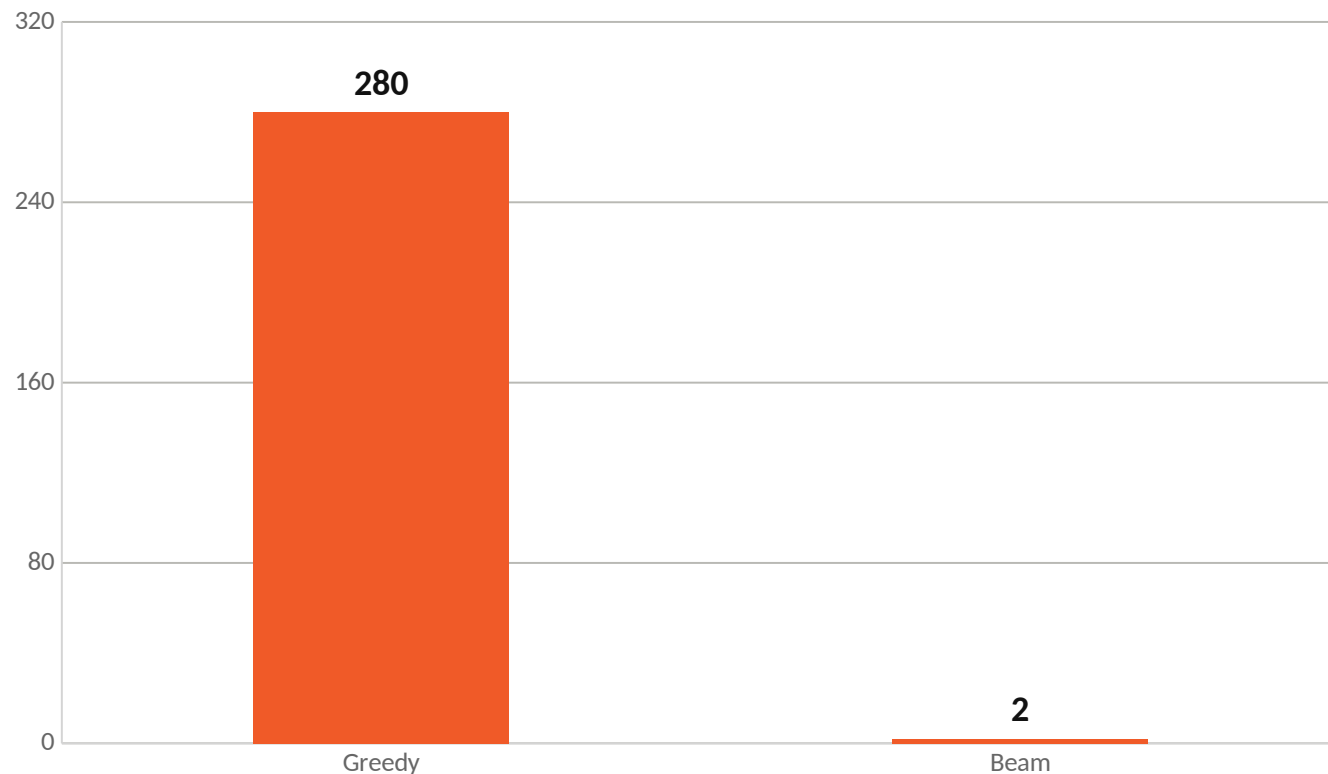
```
input_hash  
constraints  
candidate_frontier  
selected_design  
objective_upper  
objective_lower  
tie_break_rule
```



Independent verifier

- 1 восстанавливает кандидатов
- 2 повторно считает стоимость
- 3 проверяет бюджет и возможности
- 4 проверяет tie-break
-  принимает или отклоняет artifact

Beam практически совпадает с точным решением



237/240

совпадений Beam с exact

104

случая, где Beam лучше Greedy

Средний relative objective gap: 280 bp = 2,80%; 2 bp = 0,02% · 240 benchmark rows

Синтезированная структура выигрывает не всегда

9/11

CertiGap-H быстрее Fenwick

10/11

быстрее uniform-prefix

4/11

самый быстрый среди всех

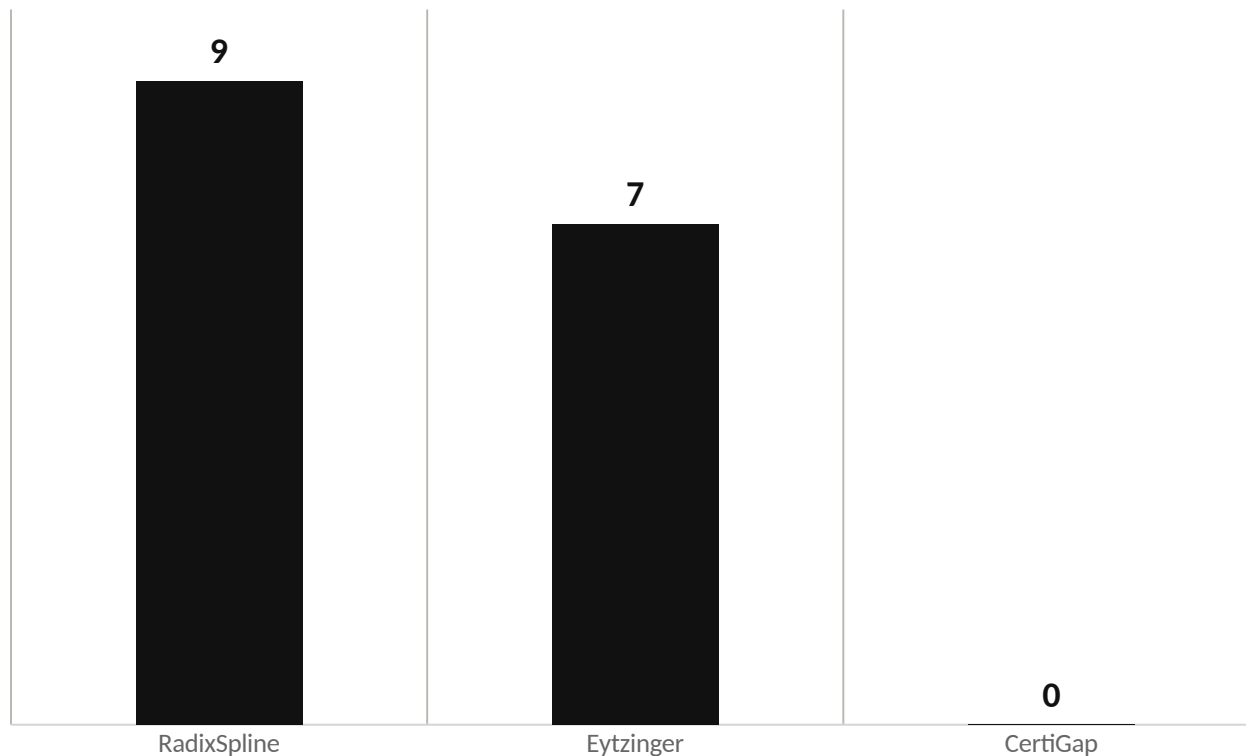
Где побеждает

read-heavy · hot/cold skew · статические каталоги

Где проигрывает

30% и 50% updates · uniform workload

На реальном поиске преимущество скромнее



10/16

побед над `std::lower_bound`

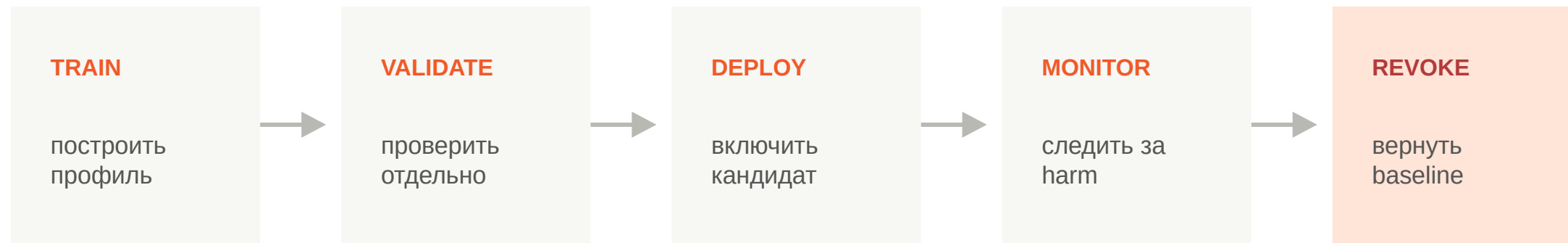
до 1,18x

лучшее ускорение

Но CertiGap ни разу не был самым быстрым.

4 SOSD distributions · 16 synthetic workload cases

Нагрузка меняется — решение должно уметь отступить



SafeAutoIndex

включает специализацию только после bounded-harm проверки

Sequential / Martingale

поддерживает многократную проверку и отзыв при доказанном вреде

Исследование доведено до исполняемого toolkit

C++17

single header
CMake package

Python

AdaptiveArray
CLI compiler

SQLite

loadable extension
virtual table

Research

paper · tests
reproduction

Минимальный путь
использования

`adaptive_array(values)` → обычные `range/update` → `explain()`

Научная новизна — в проверяемом цикле выбора

Не заявляется

«первая адаптивная структура»
«всегда быстрее Fenwick»
«глобально оптимальна везде»

Заявляется

Сертифицируемая budgeted partial materialization при ненадёжном прогнозе внутри объявленной грамматики.

Собственный вклад автора

1. Формализация задачи и cost model
2. Exact DP, beam и проверяемые границы
3. CertiGap-X/H и AutoIndex portfolio
4. Независимые verifier-ы и сертификаты
5. Реализация, тесты и воспроизводимые benchmarks

Честные границы текущей работы

Доказано

оптимум структурного score в объявленной грамматике

Измерено

latency на Apple M4 и зафиксированных workloads

Не доказано

универсальное превосходство по реальному времени

Требуется

независимая репликация Intel/AMD/ARM Linux

Требуется

официальный YCSB/RocksDB и matched learned-index baselines

ВЫВОД

CertiGap превращает выбор структуры данных в проверяемый процесс, а не ручную догадку.

Когда помогает

ограниченная память · неравномерные запросы · read-heavy workload · необходимость доказуемого выбора

Вопросы?

Источники и воспроизводимость

Правила РКНП: eGov Kazakhstan, обновлено 24.02.2026

Формат финальной защиты: открытая выставка по требованиям Regeneron ISEF

Реестр утверждений: docs/CLAIMS.md

Методы: docs/FORMALIZATION.md, SYNTHESIS.md, HYBRID.md

Результаты: results/summary.md, synthesis_native_latency.md, sosd_streaming.md

Воспроизведение: docs/REPRODUCIBILITY.md и verify_artifacts.py

Исходный код: github.com/nazkari86-lab/certigap-toolkit

Все численные утверждения в презентации связаны с зафиксированными CSV, metadata и проверяемыми artifacts.